

TRACE: Triplet Recovery via Arrowhead-Centric Extraction for Flowchart Understanding

Daozhu Dong¹, Kaiwen Shi¹, Dan Si², Xiaoyu Hao²,
Tong Liu², Wenjie Zhang², Gong Cheng¹

¹State Key Laboratory of Novel Software Technology, Nanjing University, Nanjing, China

²Lenovo (Beijing) Co., Ltd., Beijing, China

Correspondence: gcheng@nju.edu.cn

Abstract

Flowcharts are widely used in business management and software development to represent process logic and user interactions. Understanding the semantics they encode is essential for downstream applications such as visual question answering. Existing methods either detect components and reconstruct topology, which suffers from cascaded errors and rigid shape assumptions, or feed the whole image to a vision language model (VLM), which dilutes attention on dense diagrams and often reverses edge directions. Inspired by cognitive science findings that humans read flowcharts by tracing arrows, we propose TRACE (Triplet Recovery via Arrowhead-Centric Extraction), a three-stage framework. An offline stage synthesizes aligned training data from text-format flowchart sources (e.g., Mermaid, BPMN XML) and fine-tunes the two models used online: a detector that locates every arrowhead, and then a VLM that extracts one triplet per arrowhead. Each output triplet is grounded to a detected arrowhead, enabling traceable diagnosis. Experiments show that TRACE substantially improves flowchart understanding accuracy and improves downstream QA accuracy.

1 Introduction

Flowcharts have long served as a visual lingua franca for representing procedural and logical structure across domains (Bray et al., 2026): in enterprise operations, they capture workflows, approval chains, and decision trees; in software engineering, they describe call graphs, control flow, and user interactions. With the progress of VLMs, recovering the semantics of flowchart images has become foundational to downstream applications, such as visual question answering (QA) on procedural documents (Singh et al., 2024) and RAG grounded in process knowledge (de Avalor et al., 2026).

A widely adopted representation for these applications is the directed semantic triplet

(*source, condition, target*) (Bhushan et al., 2024; Deka and Devereux, 2025), where *source* and *target* name the two endpoint nodes and *condition* is the optional label on the edge (e.g., “Yes”, “if amount > 100”). However, accurately recovering the complete set of triplets from a flowchart image remains inherently challenging (Pan et al., 2024; Singh et al., 2024) due to diverse visual styles (e.g., hand-drawn sketches, hierarchical BPMN pools and lanes) and visually ambiguous arrows.

Existing flowchart understanding methods fall into two paradigms (Bray et al., 2026). *Detection-reconstruction* approaches (Liu et al., 2024b; Schäfer et al., 2021; Bhushan and Lee, 2022; Bhushan et al., 2024; Omasa et al., 2025) first detect components such as nodes, arrows, and arrow endpoints with object detectors, and then reconstruct a directed graph through spatial reasoning; even arrow-guided variants (Omasa et al., 2025) use the VLM only for downstream QA over the reconstructed graph. This paradigm suffers from compounding issues: detection errors on the node/shape vocabulary propagate into structural errors during topology assembly—a missed or misclassified shape corrupts every edge incident to it; and topology reconstruction relies on a canonical vocabulary of node shapes (e.g., rectangles, ovals), which breaks down on hand-drawn or enterprise-specific diagrams. *End-to-end* approaches (Pan et al., 2024; Singh et al., 2024; Deka and Devereux, 2025) instead feed the whole image to a VLM to emit the full structure in a single pass. While this sidesteps cascaded errors, it faces two limitations. First, attention dilutes on dense diagrams, causing individual arrows to be missed and edge directions to be frequently reversed. Second, the outputs are opaque, preventing erroneous triplets from being traced back to specific arrows for diagnosis.

Both failure modes stem from treating the flowchart as a *whole-image* understanding problem. Cognitive science suggests that humans understand

complex diagrams not by perceiving everything at once, but through localized search guided by directional cues (Larkin and Simon, 1987). Such comprehension is further facilitated when complex visual information is segmented into smaller units (Mayer, 2002). Conversely, when attention must be split across spatially dispersed sources, extraneous cognitive load is introduced and comprehension is impaired (Sweller et al., 1998).

These principles motivate us to address an analogous VLM challenge: attention dilution across spatially dispersed connectors. We therefore use sparse, readily detectable *arrowheads* as direction-bearing local anchors, one per directed edge. We instantiate this choice as TRACE (Triplet Recovery via Arrowhead-Centric Extraction), which comprises three stages: (1) an offline *data synthesis and model training* stage that derives aligned supervision from text-format flowchart sources (e.g., Mermaid, BPMN XML, PlantUML, INKML) and fine-tunes the subsequent arrowhead detector and VLM; (2) *arrowhead detection*, which localizes every arrowhead in an input image and renders one highlighted image per arrowhead; and (3) *triplet extraction*, where a VLM, guided by the highlight marker, traces back to the source node and forward to the target—along with any edge condition—to emit one triplet per image, which are aggregated into the final set. Because each triplet is tied to a specific arrow, the output is traceable and locally correctable.

Our main contributions are as follows.

- We propose TRACE, a flowchart parsing framework that recasts the task as per-arrowhead local extraction, bridging the component-detection-then-assembly and whole-image end-to-end paradigms to mitigate their limitations. It applies uniformly to various kinds of flowcharts.
- We introduce an automated data synthesis pipeline that converts text-format flowchart sources in particular languages (Mermaid, BPMN XML, PlantUML, INKML) into two aligned forms of supervision data: whole-image arrowhead bounding boxes for training the arrowhead detector and per-arrowhead annotated images for training the VLM.

We conduct extensive experiments across 9 benchmarks and 5 VLM backbones, showing

that TRACE substantially improves triplet extraction and the downstream QA accuracy. Code and data are available at <https://github.com/nju-websoft/TRACE>.

2 Related Work

From detection-reconstruction to end-to-end VLMs. Extracting structured information from diagrams underpins a range of downstream tasks, from question answering to retrieval and decision support; Bray et al. (2026) survey the broader landscape of technical diagram understanding. For flowcharts specifically, prior work falls into two paradigms. Early work framed the task as a pipeline of object detection followed by graph assembly—detecting nodes, arrows, and endpoints, then connecting them through spatial heuristics (Liu et al., 2024b; Schäfer et al., 2021; Bhushan and Lee, 2022; Bhushan et al., 2024; Arbaz et al., 2024; Omasa et al., 2025). This *detection-reconstruction* paradigm works well on diagrams with a small, canonical shape vocabulary (rectangles, ovals, diamonds) but is hard to generalize: components outside the training vocabulary—common in hand-drawn or enterprise-specific diagrams—are missed, and the downstream heuristics are easily confused by overlapping or branching edges. The rise of VLMs prompted a shift to *end-to-end* image-to-structure parsing (Pan et al., 2024; Singh et al., 2024; Deka and Devereux, 2025; Ye et al., 2025): a single VLM reads the whole image and emits the full graph as triplets or structured code (e.g., Mermaid). This sidesteps cascaded errors but introduces failure modes of its own—dense diagrams routinely yield missing or direction-reversed edges, and outputs are not grounded to any image region, so individual mistakes cannot be localized. TRACE departs from both. Unlike detection-reconstruction—including arrow-guided variants (Omasa et al., 2025) that still rely on geometric heuristics to assemble the graph—TRACE uses arrowhead detection only as a localization anchor: each arrowhead bounding box identifies a particular annotated image, and the VLM extracts the (s, c, t) triplet directly from that image. No node-shape vocabulary, contact-point matching, or other geometric assembly step is involved.

Why flowcharts are hard for VLMs. Recent probing studies clarify why the end-to-end approach struggles. The visual encoders of mainstream VLMs systematically under-represent fine-

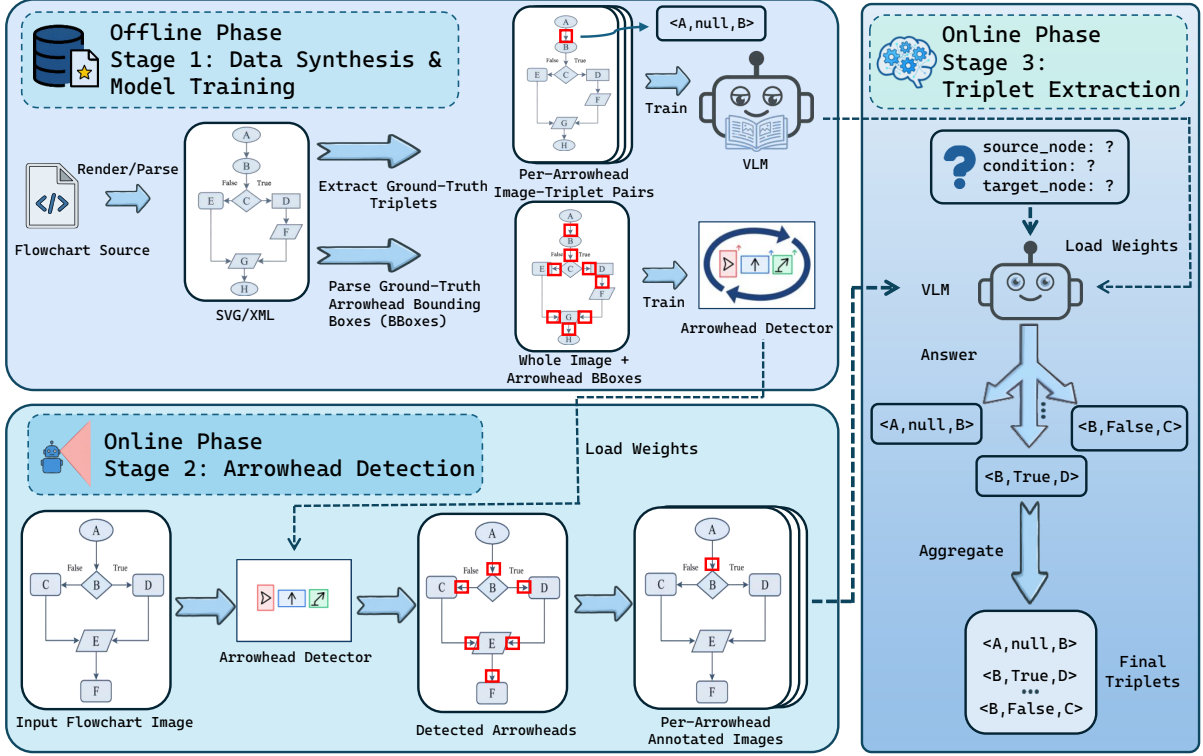


Figure 1: Overview of the TRACE framework. *Stage 1 (top-left)*: offline data synthesis from text-format flowchart sources and supervised training of the arrowhead detector and the VLM. *Stage 2 (bottom-left)*: the trained detector localizes all arrowheads in the input image and produces n per-arrowhead annotated images. *Stage 3 (right)*: the trained VLM emits a triplet for each annotated image, aggregated into the final triplet set.

grained edge and connectivity information relative to nodes (Terashita et al., 2025; Hou et al., 2025; Yoshida et al., 2026): arrows and thin connectors are encoded far more weakly than the nodes they join. A whole-image query against such an encoder is therefore poorly matched to a task whose answer hinges precisely on these under-represented connectors—the more edges a diagram packs in, the more this mismatch compounds.

Lessons from cognitive science about diagram comprehension. How humans handle the same difficulty offers an informative parallel. Readers do not perceive complex diagrams holistically; they perform localized search guided by directional cues such as arrows (Larkin and Simon, 1987). Comprehension improves when visual information is segmented into smaller units (Mayer, 2002) and degrades when attention is split across spatially dispersed sources (Sweller et al., 1998). Translating this to VLMs, Kulkarni et al. (2025) show that visually highlighting the key region of a circuit diagram markedly improves a VLM’s ability to read it—evidence that explicit visual cues, not merely better prompts, can help focus visual processing. These

observations jointly motivate TRACE’s flowchart: decompose the diagram into per-arrowhead queries (segmentation), anchor each query on a directional cue (an arrowhead), and supply an explicit highlight marker (the visual cue) to focus the encoder on one connector at a time.

3 Method

Figure 1 illustrates the three-stage pipeline of TRACE. Offline data synthesis derives aligned supervision from text-format flowchart sources and fine-tunes the arrowhead detector and the VLM that drive the two online stages (§3.2). At inference, the trained detector localizes every arrowhead in the input flowchart image and renders n per-arrowhead annotated images (§3.3); the trained VLM then emits one triplet from each image, aggregated into the final set (§3.4).

3.1 Problem Formulation

Given a flowchart image I , we represent the flowchart as a directed labeled graph $G = (V, E)$, where each node $v \in V$ carries a text label $t(v)$, and each directed edge $e = (v_s, v_t) \in E$ carries an optional condition label $c(e)$ such as “Yes”, “No”,

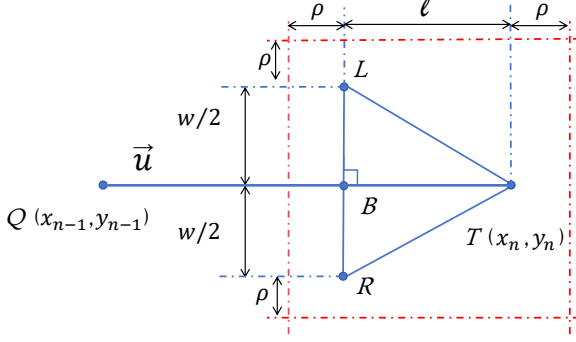


Figure 2: Geometric reconstruction of an arrowhead bounding box (red) from SVG path coordinates.

or “if amount > 100”. To support hierarchical layouts (e.g., BPMN pools and lanes), we additionally define a special relation $c(e) = \text{partOf}$ indicating that v_s is contained in the group v_t . The task is to output the triplet set

$$\mathcal{T} = \{(t(v_s), c(e), t(v_t)) \mid e = (v_s, v_t) \in E\}. \quad (1)$$

3.2 Stage 1: Data Synthesis & Model Training

To train the arrowhead detector (§3.3) and the VLM (§3.4) in the subsequent stages, we synthesize aligned supervision from text-format flowchart sources, which jointly encode topology and (directly or indirectly) visual layout. Depending on whether the source carries explicit coordinates, two pipelines are used.

Sources with Coordinates. Formats such as BPMN XML and INKML store positional information directly. We parse arrowhead bounding boxes and triplets directly from the source file.

Sources with Topology Only. Formats such as Mermaid, PlantUML, and GraphViz DOT encode only the graph structure. We first render each source into an intermediate vector graphics format, Scalable Vector Graphics (SVG). The triplets are parsed from the SVG: each arrow’s source and target nodes are identified by the LS- and LE- prefixes in the class attribute of its <path> element. The SVG, however, does not store any explicit arrowhead bounding box: each arrow appears as a <path> element with a marker-end attribute indicating that the arrow tip is anchored at the path’s terminal coordinate. We therefore reconstruct each arrowhead bounding box geometrically (Figure 2). Let $Q = (x_{n-1}, y_{n-1})$ and $T = (x_n, y_n)$ be the last two coordinates of the SVG path. The arrow

tip T is the apex of the arrowhead triangle, and the displacement from Q to T defines the unit tangent $\vec{u}$. We walk back from T along $-\vec{u}$ by an arrowhead length ℓ to obtain the base center B , and then offset B perpendicularly by $w/2$ on each side to obtain the two base corners L and R , where w is the arrowhead width. The arrowhead bounding box is the axis-aligned bounding rectangle of the triangle $\triangle TLR$, expanded by a small padding ρ . The values of ℓ , w , and ρ are given in Appendix G.

In both pipelines, we then rasterize the diagram to PNG to obtain a flowchart image I , and emit two aligned forms of supervision data:

$$\mathcal{D}_{det} = \{(I_j, \{b_k\}_{k=1}^{n_j})\}, \quad (2)$$

$$\mathcal{D}_{vlm} = \{(I_j^{(k)}, P, (s_k, c_k, t_k))\}, \quad (3)$$

where $\mathcal{D}_{det}$ pairs each whole image I_j with its arrowhead bounding boxes $\{b_k\}$ for training the arrowhead detector, and each example in $\mathcal{D}_{vlm}$ overlays a highlight marker around a single arrowhead bounding box b_k on the original image I_j to produce a new annotated image $I_j^{(k)}$, paired with a structured prompt P and the corresponding triplet. Both $\mathcal{D}_{det}$ and $\mathcal{D}_{vlm}$ are produced without manual annotation. The arrowhead detector is then fine-tuned on $\mathcal{D}_{det}$, and the VLM is fine-tuned on $\mathcal{D}_{vlm}$ to extract a triplet from a particular arrowhead. The resulting checkpoints drive the inference pipeline described in §3.3 and §3.4, respectively.

3.3 Stage 2: Arrowhead Detection

In Stage 2, the trained arrowhead detector first localizes the set of arrowhead bounding boxes $\{b_k\}_{k=1}^n$ in the input flowchart image I . For each detected bounding box b_k , we overlay a highlight marker on the flowchart image at position b_k to produce an annotated image $I^{(k)}$ that preserves the full surrounding context. This generates n per-arrowhead annotated images, processed individually in Stage 3.

3.4 Stage 3: Triplet Extraction

In Stage 3, for each per-arrowhead annotated image $I^{(k)}$, the trained VLM is queried with a structured prompt P that asks it to recover the three components of the triplet associated with the highlighted arrowhead b_k : the source node text s_k , the (optional) condition label c_k , and the target node text t_k . For diagrams with hierarchical containment, P additionally requests source and target group labels (e.g., BPMN pool / lane) to produce `partOf`

Benchmark	Style	Arrowhead BBox	QA
FlowVQA	computerized	✓	✓
FlowLearn	computerized	✓	✓
CBD	computerized	—	—
FC_A	hand-drawn	—	—
FC_B	hand-drawn	✓	—
BPMN-VLM	BPMN	✓	—
FlowGen-e/m/h	synthetic	✓	—

Table 1: The nine evaluation benchmarks. BPMN-VLM provides BPMN diagrams with pool/lane hierarchy. *Arrowhead BBox*: ground-truth arrowhead bounding boxes available. *QA*: question answering labels available.

triplets. The same procedure applies to curved arrows and overlapping or converging connectors: because each query is anchored at an arrowhead, curvature earlier along the connector does not affect the local directional cue, and overlapping arrows with distinct detected arrowheads are queried independently. When multiple branches share an overlapping arrowhead, the VLM emits multiple parallel triplets separated by “||”. Appendix E includes a correctly recovered curved back-edge. The full prompt template is given in Appendix J.

The VLM output for each arrowhead is parsed into a triplet (s_k, c_k, t_k) . The per-arrowhead outputs are aggregated into the final triplet set $\mathcal{T}$.

4 Experiments

4.1 Experimental Setup

Implementation details are in Appendix F.

4.1.1 Benchmarks

We evaluate TRACE on nine flowchart understanding benchmarks (Table 1): **FlowVQA** (Singh et al., 2024), **FlowLearn** (Pan et al., 2024), **CBD** (Bhushan and Lee, 2022), **FC_A** (Awal et al., 2011), **FC_B** (Bresler et al., 2016), **BPMN-VLM** (Deka and Devereux, 2025), and **FlowGen-e/m/h** (Shi et al., 2026) (three difficulty levels).

Except for the cross-style robustness analysis (§4.4), each model is trained and evaluated on the splits of each single benchmark, with $\mathcal{D}_{det}$ and $\mathcal{D}_{vlm}$ built according to (§3.2) on that benchmark’s own source files. We only convert the available source files into aligned supervision data, and no other flowchart images are generated or used. The split protocol and per-benchmark statistics are listed in Appendix H.

4.1.2 Backbone Models

Arrowhead Detectors. In the main experiments, the arrowhead detector in Stage 2 is instantiated with SAM 3 (Carion et al., 2025), a text-promptable detector. In the ablation study (§4.3), we replace it with YOLOv11 (Ultralytics, 2024), a closed-vocabulary one-stage detector, to assess TRACE’s sensitivity to the detector backbone.

VLMs. We use seven VLMs spanning multiple families and sizes. Five are *open-source* backbones used as the VLM in Stage 3 of TRACE and also in the baselines: Qwen3-VL-4B and Qwen3-VL-8B (Qwen Team, 2025), MiniCPM-V-4.5-8B (OpenBMB, 2025), Gemma-3-4B-IT (Gemma Team, 2025), and LLaVA-v1.6-Mistral-7B (Liu et al., 2024a). The remaining two are *large-scale* VLMs: GPT-5 (OpenAI, 2025) and GLM-4.6V (Zhipu AI, 2025).

4.1.3 Baselines

We compare TRACE against two baseline families spanning the two paradigms reviewed in §1.

End-to-end VLMs. This family feeds the whole image to a VLM (§4.1.2) that emits all triplets in one pass. (i) *Large-scale VLMs* run in a 3-shot reasoning-enabled setting prompted with the whole image to extract triplets: GPT-5 with high reasoning effort, and GLM-4.6V with thinking mode enabled. (ii) *Open-source VLMs* are evaluated both in the zero-shot setting (**ZS**) and fine-tuned (**FT**), i.e., LoRA-adapted (Hu et al., 2022) on whole images annotated with all triplets.

Detection-Reconstruction Baselines. We compare against two state-of-the-art open-source detection-reconstruction methods that assemble triplets via object detection and heuristic post-processing: BlockNet (Bhushan et al., 2024) and Flowmind2digital (Liu et al., 2024b). Each system is constrained by the fixed shape vocabulary it was trained on, so we evaluate each only on benchmarks where its released weights apply to (BlockNet: CBD, FC_B; Flowmind2digital: FC_A, FC_B, CBD), and map their outputs to our triplet evaluation protocol (Appendix K).

TextFlow (Downstream QA Baseline). For the downstream QA experiment (§4.5), we additionally compare against TextFlow (Ye et al., 2025), a state-of-the-art flowchart QA method that converts a flowchart image into graph description text (textualizer) and answers questions from that text

(reasoner). To match TRACE’s backbone and test subset, we replace TextFlow’s original reasoner—Claude 3.5 Sonnet, no longer accessible—with Qwen3-VL-4B, and bypass the textualizer by feeding the reasoner the ground-truth Mermaid code directly—a strict upper bound on any textualizer’s output. We retain TextFlow’s original protocol—the reasoner answers directly from the textual representation, without any post-processing of its textual representation. The comparison thus isolates representation (textual Mermaid vs. structured triplets) under matched backbones.

Comparison Protocol. Aligned with the end-to-end baselines, TRACE is also evaluated in ZS and FT settings. In ZS, the arrowhead detector and the VLM use pretrained weights. In FT, the arrowhead detector—SAM 3 with the fixed text prompt “arrowhead”—is fully fine-tuned on $\mathcal{D}_{det}$, and the VLM is LoRA-adapted (Hu et al., 2022) on $\mathcal{D}_{vlm}$.

For CBD and FC_A, because their flowchart sources lack concrete positions of arrowheads, the detector is not fine-tuned on these two benchmarks. Accordingly, their $\mathcal{D}_{vlm}$ is constructed by running SAM 3 to identify arrowhead bounding boxes, followed by manual verification. Further details of this benchmark-specific protocol are provided in Appendix H.

The detector’s best checkpoint is selected by the average of mAP@50 and mAP@50:95 on the validation split; VLMs (both TRACE’s VLM and the end-to-end baselines) are selected by the lowest validation loss, with early stopping and a grid search over batch sizes. Hyperparameters are listed in Appendix G.

4.1.4 Evaluation Metrics

We use F1 and rF1 for triplet extraction, and Accuracy for downstream QA.

F1. A predicted triplet (s_p, c_p, t_p) counts as a hit only if all three components exactly match a ground-truth triplet (s_g, c_g, t_g) after normalization (Appendix I). We compute set-level precision P and recall R over the predicted and ground-truth triplet sets and report $F1 = 2PR/(P + R)$.

rF1. Following FlowGen (Shi et al., 2026), relaxed F1 (rF1) additionally accepts a predicted triplet if its source, condition, and target each has edit similarity ≥ 0.85 with those of an unmatched ground-truth triplet.

Accuracy. For the downstream QA experiment, we report QA accuracy: the fraction of questions whose predicted answer exactly matches the ground-truth answer after normalization.

4.2 Comparison with Baselines

The results are summarized in Table 2.

TRACE consistently outperforms the end-to-end baselines, with the largest gains on structurally complex diagrams. TRACE (FT) achieves the best F1 and rF1 in most settings. The gap widens when whole-image understanding is most strained: on BPMN-VLM (hierarchical pool/lane), TRACE (FT) gains +22.81 pp F1 over End-to-end (FT) with Qwen3-VL-4B (87.07 vs. 64.26) and +15.90 pp with Qwen3-VL-8B; on FlowGen-h (dense topology), TRACE (FT) gains +5 pp with both Qwen3-VL backbones. By turning global parsing into independent per-arrowhead extractions, TRACE keeps each query’s visual scope bounded as the diagram grows, whereas the end-to-end baselines must allocate attention across all at once. On simpler benchmarks (CBD, FlowGen-e), End-to-end (FT) becomes competitive. A data-matched control in Appendix A shows that TRACE’s gains are not merely due to denser supervision.

TRACE (FT) with a small open-source backbone matches or exceeds large-scale VLMs. Even against 3-shot large-scale VLMs with reasoning enabled, TRACE (FT) with Qwen3-VL-4B reaches 93.77 / 87.07 / 74.82 F1 on FlowVQA / BPMN-VLM / FlowGen-h, substantially exceeding GPT-5 (85.85 / 59.82 / 34.04)—a gap of +7.92 / +27.25 / +40.78 pp. The gap is most striking on hierarchical (BPMN-VLM) and dense (FlowGen-h) diagrams, indicating that the benefit of TRACE’s per-arrowhead decomposition scales with structural complexity.

Our synthesized data effectively teaches the per-arrowhead query format. The per-arrowhead query may be novel to pretrained VLMs, so TRACE (ZS) is weak (e.g., 47.53 F1 on BPMN-VLM with Qwen3-VL-4B; 1.10 on FlowLearn with LLaVA-v1.6). Fine-tuning on $\mathcal{D}_{vlm}$ lifts these to 87.07 and 82.16, an error reduction of 75% and 82% respectively, showing that the per-arrowhead query format is readily learned from our synthesized supervision.

Detection-reconstruction baselines are less competitive. BlockNet and Flowmind2digital are de-

Method	FlowVQA		CBD		FC_A		FC_B		FlowLearn		BPMN-VLM		FlowGen-e		FlowGen-m		FlowGen-h	
	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1
<i>End-to-end large-scale VLMs (3-shot, reasoning-enabled)</i>																		
GPT-5	85.85	86.66	83.37	88.89	40.06	51.57	42.78	53.99	78.94	80.13	59.82	67.75	75.44	77.29	46.88	48.17	34.04	35.63
GLM-4.6V	86.93	87.23	81.81	86.42	38.31	45.23	49.76	57.50	60.28	71.25	56.71	60.35	76.75	79.31	45.24	46.69	37.78	39.09
<i>Detection-reconstruction baselines</i>																		
BlockNet	—	—	31.28	42.25	—	—	0.88	3.07	—	—	—	—	—	—	—	—	—	—
Flowmind2digital	—	—	7.87	9.02	2.46	4.39	2.60	6.25	—	—	—	—	—	—	—	—	—	—
<i>Qwen3-VL-4B</i>																		
TRACE (ZS)	51.91	52.39	45.19	47.82	21.67	23.13	28.84	31.65	39.89	48.05	47.53	50.23	48.06	48.74	37.18	37.52	29.62	30.33
End-to-end (ZS)	71.72	72.15	59.01	62.35	16.43	21.54	23.44	30.44	45.25	55.58	40.22	42.19	51.62	53.15	33.33	34.09	24.10	24.94
TRACE (FT)	93.77	94.47	82.56	88.83	63.91	73.24	84.32	88.94	93.61	98.25	87.07	89.39	84.18	84.95	85.42	85.68	74.82	74.95
End-to-end (FT)	92.28	92.72	83.40	88.49	64.60	73.94	80.42	87.07	88.50	96.50	64.26	66.62	85.87	86.39	77.23	77.39	68.91	69.10
<i>Qwen3-VL-8B</i>																		
TRACE (ZS)	61.60	61.90	62.00	65.80	21.01	26.88	27.22	32.90	52.25	59.31	51.77	54.50	51.74	52.09	41.18	41.44	34.29	34.41
End-to-end (ZS)	87.49	87.88	75.58	80.92	28.98	35.54	39.78	47.20	58.62	67.29	40.04	41.51	61.18	61.73	35.70	36.26	24.71	25.38
TRACE (FT)	94.38	94.77	85.27	90.52	62.36	68.68	85.35	89.59	94.52	98.27	87.08	89.49	85.00	85.31	84.44	84.79	76.21	76.26
End-to-end (FT)	92.57	92.94	84.53	89.92	63.00	70.53	80.59	88.26	91.46	96.90	71.18	73.00	85.31	85.58	77.18	77.42	70.66	70.88
<i>MiniCPM-V-4.5-8B</i>																		
TRACE (ZS)	24.29	24.70	37.17	39.68	12.45	15.41	16.19	18.95	31.56	34.35	32.77	34.33	21.97	22.95	20.58	21.34	14.41	15.03
End-to-end (ZS)	56.75	57.15	49.03	51.69	14.90	19.01	20.31	24.56	32.57	36.56	28.76	30.09	32.04	33.69	22.31	23.17	17.13	18.02
TRACE (FT)	91.03	91.55	74.63	80.06	55.81	62.80	83.68	87.48	80.51	86.55	77.58	80.58	80.90	81.83	69.84	70.53	57.60	58.24
End-to-end (FT)	88.44	88.88	77.28	81.55	39.54	48.18	67.97	71.79	76.32	82.97	61.59	65.66	76.41	77.11	63.14	63.86	52.87	53.56
<i>Gemma3-4B-IT</i>																		
TRACE (ZS)	2.22	2.41	4.41	5.12	0.52	1.30	1.21	1.70	15.22	15.85	11.29	11.64	4.85	5.69	4.43	5.11	3.28	3.82
End-to-end (ZS)	31.89	33.63	35.70	37.36	10.59	13.23	12.00	15.55	18.93	25.11	14.54	15.97	12.51	13.83	8.50	9.78	6.25	7.48
TRACE (FT)	68.88	72.48	73.84	77.53	41.86	51.76	82.20	86.84	72.50	81.62	62.70	65.99	53.50	54.83	44.48	45.60	33.16	33.88
End-to-end (FT)	80.86	83.37	69.12	72.81	39.05	49.14	69.30	73.75	69.92	79.32	49.88	51.29	46.88	47.37	28.72	29.08	20.33	20.58
<i>LLaVA-v1.6-Mistral-7B</i>																		
TRACE (ZS)	0.21	0.36	0.94	0.94	0.00	0.00	0.00	0.12	1.10	2.07	0.49	0.49	1.42	1.60	0.96	1.17	0.23	0.29
End-to-end (ZS)	9.87	12.39	30.39	32.37	3.86	5.83	7.36	9.38	8.50	14.84	1.01	1.34	5.23	5.92	3.20	3.57	1.19	1.40
TRACE (FT)	40.97	55.44	63.25	69.21	18.32	27.29	75.00	81.42	82.16	95.02	42.27	47.24	46.84	48.58	30.58	31.75	20.10	20.57
End-to-end (FT)	42.67	53.95	54.43	59.76	12.97	21.27	71.46	77.17	64.18	82.59	18.33	20.81	33.85	34.60	18.57	18.87	11.28	11.67

Table 2: Comparison with baselines. Best per (backbone, benchmark, metric) bolded.

cisively outperformed by all fine-tuned VLM methods, with both collapsing on FC_B (0.88 / 2.60 F1).

4.3 Ablation Study

To probe TRACE’s sensitivity to the arrowhead detector and whether detection errors propagate, we fix the fine-tuned VLM and vary the source of arrowhead bounding boxes: TRACE (SAM3-FT) our main version, TRACE (YOLO-FT) replacing SAM 3 with YOLOv11, TRACE (GT) using ground-truth arrowhead bounding boxes as an upper bound, and End-to-end (FT) as a baseline. CBD and FC_A are excluded due to the lack of ground-truth bounding boxes. The results are in Table 3.

TRACE is robust to the choice of arrowhead detector. TRACE (YOLO-FT) and TRACE (SAM3-FT) differ by less than 1 pp in nearly all settings—e.g., 87.09 vs. 87.07 F1 with Qwen3-VL-4B on BPMN-VLM. TRACE’s effectiveness is preserved as long as a mainstream object detector is used and fine-tuned on $\mathcal{D}_{det}$.

Fine-tuned detection approaches the oracle—detection errors barely propagate. TRACE (SAM3-FT) closely tracks TRACE (GT) (< 2 pp F1 on most benchmarks with Qwen3-VL backbones), occasionally matching it (e.g., on BPMN-VLM with MiniCPM, 77.58 vs. 77.28). In other words, arrowhead detection errors hardly propagate to Stage 3—the highlight marker needs only to indicate the arrowhead region, so the VLM tolerates loose localization. The residual gap concentrates on dense topology (e.g., FlowGen-h with Qwen3-VL-4B: 74.82 vs. 79.70).

The paradigm gap dwarfs the detector gap. With Qwen3-VL-4B on BPMN-VLM, End-to-end (FT) reaches 64.26 F1 while TRACE reaches 87.07–87.19 across detector choices— ~ 23 pp from the paradigm shift versus < 1 pp from detector variation. The same holds with weaker backbones (e.g., with LLaVA-v1.6 on BPMN-VLM: 18.33 vs. 42–43). Our arrowhead-centric framework dominates the gains over the end-to-end baselines.

Method	FlowVQA		FC_B		FlowLearn		BPMN-VLM		FlowGen-e		FlowGen-m		FlowGen-h	
	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1
<i>Qwen3-VL-4B</i>														
TRACE (SAM3-FT)	93.77	94.47	84.32	88.94	93.61	98.25	87.07	89.39	84.18	84.95	85.42	85.68	74.82	74.95
TRACE (YOLO-FT)	93.43	94.06	85.07	89.81	92.99	97.56	87.09	89.52	83.66	84.18	84.95	85.22	74.87	75.01
TRACE (GT)	93.77	95.26	83.34	88.60	94.06	98.69	87.19	89.56	85.40	86.17	88.00	88.28	79.70	79.81
End-to-end (FT)	92.28	92.72	80.42	87.07	88.50	96.50	64.26	66.62	85.87	86.39	77.23	77.39	68.91	69.10
<i>Qwen3-VL-8B</i>														
TRACE (SAM3-FT)	94.38	94.77	85.35	89.59	94.52	98.27	87.08	89.49	85.00	85.31	84.44	84.79	76.21	76.26
TRACE (YOLO-FT)	93.73	94.12	85.23	89.60	94.29	98.09	86.88	89.35	84.49	84.78	84.28	84.60	75.80	75.91
TRACE (GT)	94.13	95.23	84.43	89.16	94.66	98.58	87.35	89.77	86.26	86.41	86.56	86.81	80.79	80.96
End-to-end (FT)	92.57	92.94	80.59	88.26	91.46	96.90	71.18	73.00	85.31	85.58	77.18	77.42	70.66	70.88
<i>MiniCPM-V-4.5-8B</i>														
TRACE (SAM3-FT)	91.03	91.55	83.68	87.48	80.51	86.55	77.58	80.58	80.90	81.83	69.84	70.53	57.60	58.24
TRACE (YOLO-FT)	90.53	91.07	82.88	86.69	79.82	85.92	76.60	79.71	81.22	82.12	69.90	70.57	56.88	57.58
TRACE (GT)	91.69	93.06	82.14	85.81	80.64	86.65	77.28	80.29	82.46	83.39	71.13	71.97	59.31	59.87
End-to-end (FT)	88.44	88.88	67.97	71.79	76.32	82.97	61.59	65.66	76.41	77.11	63.14	63.86	52.87	53.56
<i>Gemma3-4B-IT</i>														
TRACE (SAM3-FT)	68.88	72.48	82.20	86.84	72.57	81.63	62.70	65.99	53.50	54.83	44.48	45.60	33.16	33.88
TRACE (YOLO-FT)	72.21	76.15	81.93	86.12	74.59	83.98	62.49	65.85	53.95	55.45	45.66	46.82	34.28	34.93
TRACE (GT)	72.64	79.27	80.35	85.53	80.21	90.78	63.39	66.93	54.98	56.03	43.40	44.26	30.79	31.44
End-to-end (FT)	80.86	83.37	69.30	73.75	69.92	79.32	49.88	51.29	46.88	47.37	28.72	29.08	20.33	20.58
<i>LLaVA-v1.6-Mistral-7B</i>														
TRACE (SAM3-FT)	40.97	55.44	75.00	81.42	82.16	95.02	42.27	47.24	46.84	48.58	30.58	31.75	20.10	20.57
TRACE (YOLO-FT)	41.25	56.00	75.72	82.12	82.15	95.21	42.63	47.58	47.71	49.56	31.27	32.34	19.10	19.59
TRACE (GT)	32.78	44.61	75.16	82.08	82.91	95.32	42.70	47.84	44.00	45.91	27.79	28.85	16.25	16.69
End-to-end (FT)	42.67	53.95	71.46	77.17	64.18	82.59	18.33	20.81	33.85	34.60	18.57	18.87	11.28	11.57

Table 3: Ablation study: we vary the arrowhead detector. Best per (backbone, benchmark, metric) bolded.

4.4 Cross-Style Robustness

§4.2–4.3 evaluate TRACE in an in-domain setting. We now test how each paradigm transfers to an unseen flowchart style. Using a leave-one-out (LOO) protocol, we fine-tune on the union of eight benchmarks’ training splits and evaluate on the held-out one. Across this protocol, TRACE LOO outperforms End-to-end LOO in F1 on 8/9 benchmarks and in rF1 on all 9, and surpasses GPT-5 on F1 for 6/9 benchmarks, with margins up to ~ 35 pp on dense diagrams. Full results and per-benchmark analysis are in Appendix B.

The held-out style’s difficulty varies with how well the training pool covers it: when similar styles are seen (e.g., FlowGen-h alongside FlowGen-e/m), transfer is strong; when the held-out style is unique in format (FC_B, FlowLearn, BPMN-VLM), all paradigms degrade—but End-to-end LOO degrades more than TRACE LOO under the same pool, so TRACE retains its margin throughout. This isolates the gain to our task decomposition rather than a style-agnostic representation: the per-arrowhead operation (“trace back from a highlighted arrowhead to its source and target”) depends less on style than whole-image extraction.

4.5 Downstream QA Evaluation

We compare TRACE with tool-call reasoning over extracted triplets against TextFlow on

Methods	FlowLearn	FlowVQA
Qwen3-VL-4B (ZS)	66.83	62.03
TextFlow w/ ground-truth text	88.33	75.18
TRACE + tool-call	98.17	75.78
GT triplets + tool-call (oracle)	100.00	78.46

Table 4: QA accuracy on TextFlow’s released subset.

TextFlow’s released QA subset (197 FlowVQA / 100 FlowLearn images), both using Qwen3-VL-4B as the reasoner backbone. Configurations: **TRACE + tool-call**—the reasoner selects and calls graph-query tools (Appendix L) over TRACE-extracted triplets; **TextFlow w/ ground-truth text**—TextFlow’s reasoner fed with ground-truth Mermaid code (§4.1.3); **Qwen3-VL-4B (ZS)** as an end-to-end baseline; and **GT triplets + tool-call** as an oracle upper bound, feeding the reasoner with ground-truth triplets. FlowVQA answers are scored by LLM-as-judge (GLM-4.6V, Appendix M) using TextFlow’s verification prompt; FlowLearn answers the reasoner exact string match. The results are shown in Table 4.

TRACE outperforms TextFlow under matched backbone. On FlowLearn, TRACE reaches 98.17% vs. 88.33% for TextFlow given the ground-truth Mermaid code (+9.84 pp), confirming the gap is not related to TextFlow’s textualization: TRACE’s structured triplets allow the same back-

bone to answer questions more accurately via direct graph queries (e.g., counting edges) rather than from text. On FlowVQA, TRACE narrowly leads (75.78 vs. 75.18, +0.60 pp). The much smaller margin is expected: many FlowVQA questions fall into the open-ended *applied-scenario* category, whose answers require natural language reasoning over node text rather than graph topology, so structured triplets offer less leverage. Holding the reasoner fixed, TRACE with its extracted triplets approaches its variant using ground-truth triplets, demonstrating TRACE’s extraction quality.

4.6 Error Analysis

We sample 50 images per benchmark and classify each error made by TRACE (FT) with Qwen3-VL-4B into six categories (per-benchmark breakdown in Appendix C). Spurious (33.8%) and missing (27.4%) triplets dominate overall. Both stem from the same cause—the VLM mistraces an edge when nearby edges intersect or the edge color blends into the background. In contrast, direction reversal—a frequent failure of end-to-end VLMs (Shi et al., 2026)—accounts for only 10.4% of TRACE’s errors, indicating the per-arrowhead anchoring largely resolves it.

Crucially, this failure scales with edge density, not the complexity of the encoded content. On FlowGen-e/m/h, which progressively increase edge density while keeping the underlying generation process unchanged, spurious and missing triplets together account for a growing share of errors—from 17.6% (e) to 63.5% (m) to 83.2% (h). Beyond density, the dominant error type also shifts with diagram style. FC_A is dominated by wrong-node errors (42.0%, for blurred shape boundaries), whereas FlowLearn shows almost no spurious or missing triplets (0.0% / 8.5%)—its structures are recovered correctly—but instead concentrates on direction and character errors (46.8% / 44.7%), as its out-of-order character-sequence node text resists transcription.

4.7 Inference Latency

TRACE’s per-arrowhead decomposition delivers higher extraction accuracy (§4.2) at the cost of prompting the VLM once per detected arrowhead, which is slower than the single-call end-to-end baseline on most benchmarks. With Qwen3-VL-4B on NVIDIA RTX 5880 Ada, TRACE averages $2.14\times$ the per-image latency across the nine benchmarks. Interestingly, on BPMN-VLM the ratio

inverts: TRACE is faster ($0.53\times$), because a single per-arrowhead prompt is far shorter than enumerating an entire BPMN diagram in one pass. Multi-arrowhead batching reduces this overhead: increasing the number K of arrowheads per query from 1 to 3 lowers macro-average latency from 4.00 to 2.44 seconds while only reducing macro F1 from 83.30 to 82.81. This is comparable to End-to-end (FT)’s 2.73 seconds (Appendix D).

4.8 Per-Arrowhead Traceability

An additional advantage of TRACE is that each output triplet is bound to a specific arrowhead, so erroneous triplets can be localized to a single arrowhead and re-queried in isolation. Appendix E illustrates this with a case study on FlowVQA.

5 Conclusion

We present TRACE, a three-stage framework that recasts flowchart parsing as per-arrowhead local extraction, supported by an offline data synthesis pipeline that produces aligned supervision for the arrowhead detector and the VLM. Across nine benchmarks and five backbones, fine-tuned TRACE consistently outperforms end-to-end VLM baselines, with a 4B open-source backbone surpassing GPT-5 on most benchmarks. The extracted triplets also substantially boost downstream QA accuracy. Because each output triplet is grounded to a specific arrowhead, TRACE also has the potential to support human-in-the-loop correction, making it practical for deployments in regulated workflow digitization and compliance auditing.

Per-arrowhead granularity also enables targeted sample reweighting: heterogeneous errors concentrate on hard cases (§4.6), where per-arrowhead difficulty provides a more informative signal than uniform image-level supervision.

Acknowledgments

This work was sponsored by CCF-Lenovo Blue Ocean Research Fund and supported by the Fundamental Research Funds for the Central Universities (No. 2026300429) and the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118).

Limitations

We note two limitations of our work.

(1) **Undirected edges.** Our current implementation targets directed flowcharts with explicit arrowheads. We cannot empirically validate undirected edges because, to our knowledge, no dedicated flowchart benchmark with undirected node-edge triplet annotations currently exists. Extending TRACE should be straightforward: replace the arrowhead detector with a detector for generic connection endpoints, then feed the endpoint-annotated image to the VLM as before. Since an undirected connection has two endpoints, tracing from either one should be sufficient to recover the corresponding triplet.

(2) **Marker sensitivity.** TRACE relies on the VLM perceiving the per-arrowhead highlight marker. This can make performance backbone-dependent: for example, with the fixed-resolution Gemma3-4B-IT backbone on FlowVQA, End-to-end (FT) outperforms TRACE (FT) because down-sampling can weaken small visual cues. However, recent VLMs typically adopt dynamic resolution, which better preserves the marker and is more compatible with our design: for example, with Qwen3-VL-4B, Qwen3-VL-8B, and MiniCPM-V-4.5-8B on FlowVQA, TRACE (FT) outperforms End-to-end (FT).

References

- Abdul Arbaz, Heng Fan, Junhua Ding, Meikang Qiu, and Yunhe Feng. 2024. [GenFlowchart: Parsing and understanding flowchart using generative AI](#). In *Knowledge Science, Engineering and Management*, volume 14884 of *Lecture Notes in Computer Science*, pages 99–111. Springer.
- Ahmad-Montaser Awal, Guihuan Feng, Harold Mouchère, and Christian Viard-Gaudin. 2011. [First experiments on a new online handwritten flowchart database](#). In *Document Recognition and Retrieval XVIII, part of the IS&T-SPIE Electronic Imaging Symposium, San Jose, CA, USA, January 26-27, 2011, Proceedings*, SPIE Proceedings, page 78740A. SPIE.
- Shreyanshu Bhushan, Eun-Soo Jung, and Minho Lee. 2024. [Unveiling the power of integration: Block diagram summarization through local-global fusion](#). In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024, Findings of ACL*, pages 13837–13856. Association for Computational Linguistics.
- Shreyanshu Bhushan and Minho Lee. 2022. [Block Diagram-to-Text: Understanding block diagram images by generating natural language descriptors](#). In *Findings of the Association for Computational Linguistics: AACL-IJCNLP 2022, Findings of ACL*, pages 153–168. Association for Computational Linguistics.
- Nick Bray, Michael Hempel, Matthew Boeding, and Hamid Sharif. 2026. [Decoding technical diagrams: A survey of AI methods for image content extraction and understanding](#). *Inf.*, 17(2):165.
- Martin Bresler, Daniel Průša, and Václav Hlavác. 2016. [Online recognition of sketched arrow-connected diagrams](#). *Int. J. Document Anal. Recognit.*, 19(3):253–267.
- Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, and 19 others. 2025. [SAM 3: Segment anything with concepts](#). *Preprint*, arXiv:2511.16719.
- Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. [End-to-end object detection with transformers](#). In *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part I*, Lecture Notes in Computer Science, pages 213–229. Springer.
- Guillermo Gil de Avalle, Laura Maruster, and Christos Emmanouilidis. 2026. [Procedural knowledge extraction from industrial troubleshooting guides using vision language models](#). *CoRR*, abs/2601.22754.
- Pritam Deka and Barry Devereux. 2025. [Structured extraction from business process diagrams using vision-language models](#). *CoRR*, abs/2511.22448.
- Gemma Team. 2025. [Gemma 3 technical report](#). *CoRR*, abs/2503.19786.
- Yifan Hou, Buse Giledereli, Yilei Tu, and Mrinmaya Sachan. 2025. [Do vision-language models really understand visual language?](#) In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025, Proceedings of Machine Learning Research*. PMLR / OpenReview.net.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [Lora: Low-rank adaptation of large language models](#). In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.
- Shreyas Kulkarni, Vivek Kumar, Remish Leonard Minz, Munender Varshney, Thiruvengadam Samon, Abhishek Mitra, Nikhil Kulkarni, Nilanjan Chakravorty, Prateek Mital, and Kingshuk Banerjee. 2025. [Enhancing circuit diagram understanding via near sight correction using vlms](#). In *IEEE/CVF International*

- Conference on Computer Vision, ICCV 2025 - Workshops, Honolulu, HI, USA, October 19-20, 2025*, pages 4234–4242. IEEE.
- Jill H. Larkin and Herbert A. Simon. 1987. [Why a diagram is \(sometimes\) worth ten thousand words](#). *Cogn. Sci.*, 11(1):65–100.
- Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. 2024a. LLaVA-NeXT: Improved reasoning, OCR, and world knowledge. <https://llava-vl.github.io/blog/2024-01-30-llava-next/>.
- Huanyu Liu, Jianfeng Cai, Tingjia Zhang, Hongsheng Li, Siyuan Wang, Guangming Zhu, Syed Afaq Ali Shah, Mohammed Bennamoun, and Liang Zhang. 2024b. [Flowmind2digital: The first comprehensive flowmind recognition and conversion approach](#). *CoRR*, abs/2401.03742.
- Richard E Mayer. 2002. [Multimedia learning](#). In *Psychology of learning and motivation*, volume 41, pages 85–139. Elsevier.
- Takamitsu Omasa, Ryo Koshihara, and Masumi Morishige. 2025. [Arrow-guided VLM: enhancing flowchart understanding via arrow direction encoding](#). *CoRR*, abs/2505.07864.
- OpenAI. 2025. GPT-5. <https://openai.com/index/introducing-gpt-5/>.
- OpenBMB. 2025. MiniCPM-V 4.5: Efficient MLLM for single image, multi-image and high-FPS video understanding. https://huggingface.co/openbmb/MiniCPM-V-4_5.
- Huitong Pan, Qi Zhang, Cornelia Caragea, Edward C. Dragut, and Longin Jan Latecki. 2024. [Flowlearn: Evaluating large vision-language models on flowchart understanding](#). In *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, Frontiers in Artificial Intelligence and Applications, pages 73–80. IOS Press.
- Qwen Team. 2025. [Qwen3-vl technical report](#). *CoRR*, abs/2511.21631.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. [Zero: memory optimizations toward training trillion parameter models](#). In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, page 20. IEEE/ACM.
- Hamid Reza Tofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian D. Reid, and Silvio Savarese. 2019. [Generalized intersection over union: A metric and a loss for bounding box regression](#). In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 658–666. Computer Vision Foundation / IEEE.
- Bernhard Schäfer, Margret Keuper, and Heiner Stuckenschmidt. 2021. [Arrow R-CNN for handwritten diagram recognition](#). *Int. J. Document Anal. Recognit.*, 24(1):3–17.
- Kaiwen Shi, Sichen Liu, Ziyue Lin, Hangrui Guo, and Gong Cheng. 2026. [Flowgen: Synthesizing diverse flowcharts to enhance and benchmark mllm reasoning](#). In *The Fourteenth International Conference on Learning Representations*.
- Shubhankar Singh, Purvi Chaurasia, Yerram Varun, Pranshu Pandya, Vatsal Gupta, Vivek Gupta, and Dan Roth. 2024. [Flowvqa: Mapping multimodal logic in visual question answering with flowcharts](#). In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Findings of ACL, pages 1330–1350. Association for Computational Linguistics.
- John Sweller, Jeroen JG Van Merriënboer, and Fred GWC Paas. 1998. [Cognitive architecture and instructional design](#). *Educational psychology review*, 10(3):251–296.
- Naoyuki Terashita, Yusuke Tozaki, Hideaki Omote, Congkha Nguyen, Ryosuke Nakamoto, Yuta Koreeda, and Hiroaki Ozaki. 2025. [Can visual encoder learn to see arrows?](#) *CoRR*, abs/2505.19944.
- Ultralytics. 2024. YOLO11. <https://github.com/ultralytics/ultralytics>.
- Junyi Ye, Ankan Dash, Wenpeng Yin, and Guiling Wang. 2025. [Beyond end-to-end vlms: Leveraging intermediate text representations for superior flowchart understanding](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, pages 3534–3548. Association for Computational Linguistics.
- Haruto Yoshida, Keito Kudo, Yoichi Aoki, Ryota Tanaka, Itsumi Saito, Keisuke Sakaguchi, and Kentaro Inui. 2026. [Nodes are early, edges are late: Probing diagram representations in large vision-language models](#). *CoRR*, abs/2603.02865.
- Zhipu AI. 2025. GLM-4.6V. <https://huggingface.co/zai-org/GLM-4.6V>.

A Data-Matched Control

Setup. We add a data-matched control for End-to-end (FT) with Qwen3-VL-4B. For each benchmark, every image-level training sample is repeated R times, where R is the ratio of TRACE’s arrow-level samples to image-level samples. This matches the per-epoch training-instance scale induced by per-arrowhead decomposition, while all other training and evaluation settings follow the main experiments, including validation-loss early stopping for update selection.

Results. Repetition does not provide per-arrowhead localization cues. Table 5 shows that TRACE still outperforms this data-matched baseline in F1 on 8/9 benchmarks and in rF1 on all 9, so the gains are not explained merely by denser supervision.

B Cross-Style Robustness Analysis

To probe cross-style robustness, i.e., how each paradigm degrades when a target style is unseen during training—we run a leave-one-out (LOO) protocol with Qwen3-VL-4B: for each target benchmark D , we fine-tune on the union of the other eight benchmarks’ training splits and evaluate on D ’s test set. We compare TRACE LOO against TRACE IN-DOM (i.e., TRACE (FT) in Table 2), **End-to-end LOO** (the same LOO protocol on the end-to-end baseline), and **GPT-5**. Full results are in Table 6.

LOO training configuration. The leave-one-out experiments reuse the main-experiment recipe (Appendix G: LoRA rank, learning rate, scheduler, optimizer, etc.) with one change: each LOO run trains for 1 epoch instead of 10. The batch size is set to the most frequent optimal value across the other benchmarks’ main-experiment runs.

Composition of the LOO training pool. The two paradigms consume training data at different levels of granularity: TRACE sees one per-arrowhead sample (an element of $\mathcal{D}_{vlm}$), while the end-to-end baseline sees one image with its full triplet list. The two pools therefore have different size distributions across the nine benchmarks (Table 7). This difference follows from the per-arrowhead task decomposition: both paradigms are trained on exactly the same source images and triplets, but TRACE localizes each supervision target to a specific arrowhead.

TRACE retains its margin under the same cross-style protocol. Across the nine benchmarks, TRACE LOO outperforms End-to-end LOO in F1 on 8/9 (except for CBD, -0.35 pp) and in rF1 on all 9, and surpasses GPT-5 in F1 on 6/9 benchmarks. The relative gain is largest where cross-style transfer is hardest: on FlowGen-h and FlowGen-m, TRACE LOO leads End-to-end LOO by up to ~ 14 pp and GPT-5 by up to ~ 35 pp. The ordering against End-to-end LOO is preserved on every benchmark—TRACE never loses to end-to-end under the identical training pool—supporting the interpretation that the per-arrowhead task decomposition is what carries through, not a style-agnostic representation.

With a stylistic neighbor in the pool, TRACE LOO approaches TRACE IN-DOM. On five of nine benchmarks, TRACE LOO stays within ~ 6 pp F1 of TRACE IN-DOM (CBD, FC_A, FlowVQA, FlowGen-m, FlowGen-h); on FlowGen-e it exceeds TRACE IN-DOM by 12.79 pp, because the eight-benchmark training pool dwarfs FlowGen-e’s own training set. When similar styles are seen (e.g., FlowGen-h alongside FlowGen-e/m), transfer is strong.

Without a stylistic neighbor, absolute drops are large but the relative gain persists. FC_B (-16.56), FlowLearn (-27.68), and BPMN-VLM (-47.89) are the three largest drops from TRACE LOO to TRACE IN-DOM. FC_B’s drop comes from data scarcity: when it is held out, the only other hand-drawn benchmark (FC_A) contributes just $\sim 1\text{--}2\%$ of the LOO training pool (Table 7). FlowLearn and BPMN-VLM are style outliers in different ways—FlowLearn’s node text is an out-of-order character sequence (more like visual tokens than natural words), and many BPMN nodes carry no text labels at all. On these three benchmarks, TRACE LOO still outperforms End-to-end LOO under the same training pool, but neither paradigm can recover TRACE IN-DOM performance: a style whose output form does not appear in the training pool is an inherent limit of any cross-style flowchart parsing setup, not a TRACE-specific weakness. For FlowLearn, an OCR pass on its node text (detailed below) raises TRACE LOO from 65.93 to 92.43 F1 and End-to-end LOO from 53.77 to 76.99; for FC_B and BPMN-VLM, no surface-form refinement applies cleanly without drawing on in-domain data.

The remainder of this appendix details the

Method	FlowVQA		CBD		FC_A		FC_B		FlowLearn		BPMN-VLM		FlowGen-e		FlowGen-m		FlowGen-h	
	$R=25$		$R=7$		$R=10$		$R=7$		$R=8$		$R=22$		$R=10$		$R=21$		$R=35$	
	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1	F1	rF1
TRACE (FT)	93.77	94.47	82.56	88.83	63.91	73.24	84.32	88.94	93.61	98.25	87.07	89.39	84.18	84.95	85.42	85.68	74.82	74.95
End-to-end (FT)	92.28	92.72	83.40	88.49	64.60	73.94	80.42	87.07	88.50	96.50	64.26	66.62	85.87	86.39	77.23	77.39	68.91	69.10
End-to-end (FT) data-matched	91.04	91.67	77.65	83.37	34.81	43.46	71.25	76.89	88.80	97.48	67.66	70.07	84.66	84.91	78.33	78.55	66.99	67.23

Table 5: Data-matched control using Qwen3-VL-4B. R is the ratio of TRACE’s arrow-level samples to image-level samples. Best bolded.

Benchmark	TRACE IN-DOM		TRACE LOO		End-to-end LOO		GPT-5	
	F1	rF1	F1	rF1	F1	rF1	F1	rF1
FlowVQA	93.77	94.47	89.31	90.28	87.20	87.68	85.85	86.66
CBD	82.56	88.83	83.31	88.36	83.66	87.94	83.37	88.89
FC_A	63.91	73.24	61.38	71.32	45.46	52.66	40.06	51.57
FC_B	84.32	88.94	67.76	75.31	57.26	64.23	42.78	53.99
FlowLearn	93.61	98.25	65.93 (92.43)	89.69 (92.61)	53.77 (76.99)	75.05 (77.15)	78.94	80.13
BPMN-VLM	87.07	89.39	39.18	41.08	31.86	33.99	59.82	67.75
FlowGen-e	84.18	84.95	96.97	97.32	95.87	96.30	75.44	77.29
FlowGen-m	85.42	85.68	79.55	79.63	65.21	65.49	46.88	48.17
FlowGen-h	74.82	74.95	69.42	69.57	57.93	58.20	34.04	35.63

Table 6: Cross-style robustness analysis. Numbers in parentheses report + **Post**, an OCR pass on FlowLearn node text. Best bolded.

Benchmark	TRACE ($\mathcal{D}_{vlm}$)		End-to-end (images)	
	#	%	#	%
FlowVQA	29,586	19.3	1,187	9.9
FlowLearn	56,067	36.5	7,200	60.2
CBD	2,011	1.3	300	2.5
FC_A	1,842	1.2	191	1.6
FC_B	1,898	1.2	280	2.3
BPMN-VLM	2,193	1.4	100	0.8
FlowGen-e	9,423	6.1	900	7.5
FlowGen-m	18,890	12.3	900	7.5
FlowGen-h	31,721	20.7	900	7.5
Total	153,631	100.0	11,958	100.0

Table 7: Per-benchmark share of the LOO training pool, as seen by TRACE (per-arrowhead $\mathcal{D}_{vlm}$ samples) and the end-to-end baseline (whole-image triplet-list pairs). When a benchmark D is held out, the LOO training pool is the union of the other eight rows’ training splits.

FlowLearn OCR refinement applied to both End-to-end LOO and TRACE LOO in the + **Post** columns of Table 6.

FlowLearn OCR pass. FlowLearn’s node text consists of out-of-order character sequences that the LOO-trained VLM rarely reproduces verbatim. We rescue them with an OCR-vocabulary edit distance correction:

1. Run PaddleOCR (PP-OCRv5, English preset) on the input image to obtain a vocabulary of

recognized tokens (no score thresholding; min confidence 0).

2. For each component $f \in \{\text{source, end, condition}\}$ of each predicted triplet, compute the case-insensitive Levenshtein distance from f to every token in the OCR vocabulary; if a *unique* token within distance ≤ 2 exists, replace f with that token. Otherwise keep f unchanged.
3. A small set of reserved relation tokens (connectedTo, partOf, connected_with) are never replaced, since the relation vocabulary is not OCR-derived.

C Error Analysis Details

For each of the nine benchmarks, we randomly sample 50 images (seed=42) and run TRACE (FT) with Qwen3-VL-4B to obtain predicted triplets. Predicted and ground-truth triplets are normalized (lowercased, punctuation stripped, whitespace removed). For each predicted triplet (s, c, t) we greedily assign it to the unmatched ground-truth triplet with the highest (s, t) -similarity (Levenshtein, threshold 0.5), allowing either the forward (s, t) or reverse (t, s) direction. Each triplet is then classified into one of six error categories (or counted as correct):

Benchmark	#	Miss.	Spur.	Dir.	Node	Char.	Cond.
FlowVQA	144	21.5	47.2	13.2	9.7	7.6	0.7
CBD	102	18.6	15.7	19.6	1.0	32.4	12.8
FC_A	193	31.1	14.5	2.1	42.0	6.2	4.2
FC_B	75	12.0	14.7	25.3	26.7	17.3	4.0
FlowLearn	47	8.5	0.0	46.8	0.0	44.7	0.0
BPMN-VLM	414	33.6	38.9	10.1	4.8	8.5	4.1
FlowGen-e	85	4.7	12.9	12.9	0.0	5.9	63.5
FlowGen-m	159	17.6	45.9	7.5	10.7	5.0	13.2
FlowGen-h	375	37.9	45.3	4.5	1.9	0.0	10.4
ALL	1594	27.4	33.8	10.4	10.0	8.7	9.8

Table 8: Per-benchmark error distribution (% of errors in each row) for TRACE (FT) with Qwen3-VL-4B. The dominant category in each row is bolded.

Benchmark	End-to-end (s)	TRACE (s)	Ratio
FC_A	0.52	0.81	1.56×
CBD	0.64	0.72	1.13×
FC_B	0.47	0.96	2.04×
FlowLearn	0.53	1.03	1.94×
FlowVQA	1.96	11.41	5.82×
BPMN-VLM	9.50	5.04	0.53 ×
FlowGen-e	0.51	1.56	3.06×
FlowGen-m	2.33	4.51	1.94×
FlowGen-h	8.11	9.96	1.23×

Table 9: Per-image inference latency (Qwen3-VL-4B, NVIDIA RTX 5880 Ada). Ratio = TRACE / end-to-end. Lower is better for TRACE.

- *missing*: ground-truth triplet with no predicted match.
- *spurious*: not matched with any ground-truth triplet.
- *direction*: only matched with a ground-truth triplet in the reverse direction with both node similarities ≥ 0.85 .
- *node*: matched but at least one node has similarity < 0.85 .
- *character*: matched and both nodes have similarity ≥ 0.85 but at least one differs by 1–2 characters.
- *condition*: both nodes matched exactly but the condition label differs.

Table 8 reports the per-benchmark counts and within-row percentages.

D Inference Latency

This appendix reports per-image inference latency for TRACE and the end-to-end baseline (Table 9), and evaluates multi-arrowhead batching across all nine benchmarks (Table 10). All measurements use the same configuration as the main experiments

Benchmark	K	F1	rF1	Latency (s)	Ratio
FC_A	E2E	64.60	73.94	0.52	1.00×
	1	63.91	73.24	0.81	1.56×
	2	62.93	72.35	0.83	1.60×
	3	62.71	72.27	0.78	1.50×
CBD	E2E	83.40	88.49	0.64	1.00×
	1	82.56	88.83	0.72	1.13×
	2	82.43	87.98	0.52	0.81×
	3	80.91	86.38	0.49	0.77×
FC_B	E2E	80.42	87.07	0.47	1.00×
	1	84.32	88.94	0.96	2.04×
	2	84.07	89.26	0.81	1.72×
	3	83.79	88.26	0.77	1.64×
FlowLearn	E2E	88.50	96.50	0.53	1.00×
	1	93.61	98.25	1.03	1.94×
	2	93.38	98.16	0.55	1.04×
	3	94.08	98.87	0.54	1.02×
FlowVQA	E2E	92.28	92.72	1.96	1.00×
	1	93.77	94.47	11.41	5.82×
	2	93.10	93.92	5.83	2.97×
	3	93.25	94.01	4.58	2.34×
BPMN-VLM	E2E	64.26	66.62	9.50	1.00×
	1	87.07	89.39	5.04	0.53×
	2	86.15	88.24	3.31	0.35×
	3	86.78	88.63	3.17	0.33×
FlowGen-e	E2E	85.87	86.39	0.51	1.00×
	1	84.18	84.95	1.56	3.06×
	2	85.36	85.84	1.13	2.22×
	3	85.40	85.69	1.08	2.12×
FlowGen-m	E2E	77.23	77.39	2.33	1.00×
	1	85.42	85.68	4.51	1.94×
	2	84.78	84.78	3.65	1.57×
	3	84.72	84.72	3.28	1.41×
FlowGen-h	E2E	68.91	69.10	8.11	1.00×
	1	74.82	74.95	9.96	1.23×
	2	74.10	74.18	8.43	1.04×
	3	73.65	73.76	7.23	0.89×
Macro Avg.	E2E	78.39	82.02	2.73	1.00×
	1	83.30	86.52	4.00	2.14×
	2	82.92	86.08	2.78	1.48×
	3	82.81	85.84	2.44	1.33×

Table 10: Multi-arrowhead batching on all nine benchmarks. E2E rows reproduce End-to-end (FT) from Table 2; $K=1$ rows reproduce the main TRACE (FT) configuration. Ratio denotes latency relative to the E2E baseline for each benchmark. Macro Avg. is the arithmetic mean across benchmarks; for Ratio, Macro Avg. is the mean of per-benchmark ratios.

(§4): Qwen3-VL-4B as the VLM, paired with the arrowhead detector from the main experiments, on NVIDIA RTX 5880 Ada hardware.

Multi-arrowhead batching. We evaluate on all nine benchmarks with $K \in \{1, 2, 3\}$, where K is the number of arrowheads highlighted on the

same input image in a single VLM query (the main configuration is $K=1$, one arrowhead per query). The prompt asks the VLM to output the triplet corresponding to each highlighted arrowhead. A separate LoRA adapter is trained for each K to match the inference-time annotation density.

The K -batching mechanism is benchmark-agnostic: each annotated image carries up to K highlight markers and the prompt asks for K aligned triplets, regardless of the underlying benchmark. Across all nine benchmarks, increasing K from 1 to 3 reduces macro-average latency from 4.00 to 2.44 seconds and the mean ratio relative to E2E from $2.14\times$ to $1.33\times$. The resulting macro-average latency is comparable to that of E2E (2.44 vs. 2.73 seconds). Macro F1 decreases by only 0.49 points (83.30 to 82.81). At $K=3$, TRACE remains 4.42 points above E2E in macro F1 (82.81 vs. 78.39), at a mean latency ratio of $1.33\times$.

E Case Study: Per-Arrowhead Traceability

This case study illustrates an additional advantage of TRACE: every output triplet is bound to a specific arrowhead, so users can locate, inspect, and locally correct the model’s predictions. We use one FlowVQA example (code00005; Figures 3a and 3b). SAM 3 detects ten arrowheads (**H1**–**H10**, top-to-bottom, left-to-right) and each one yields one triplet from the VLM; the end-to-end baseline emits a flat list of triplets with no link back to specific arrowheads.

TRACE output. Each triplet is bound to a specific arrowhead label from Figure 3b:

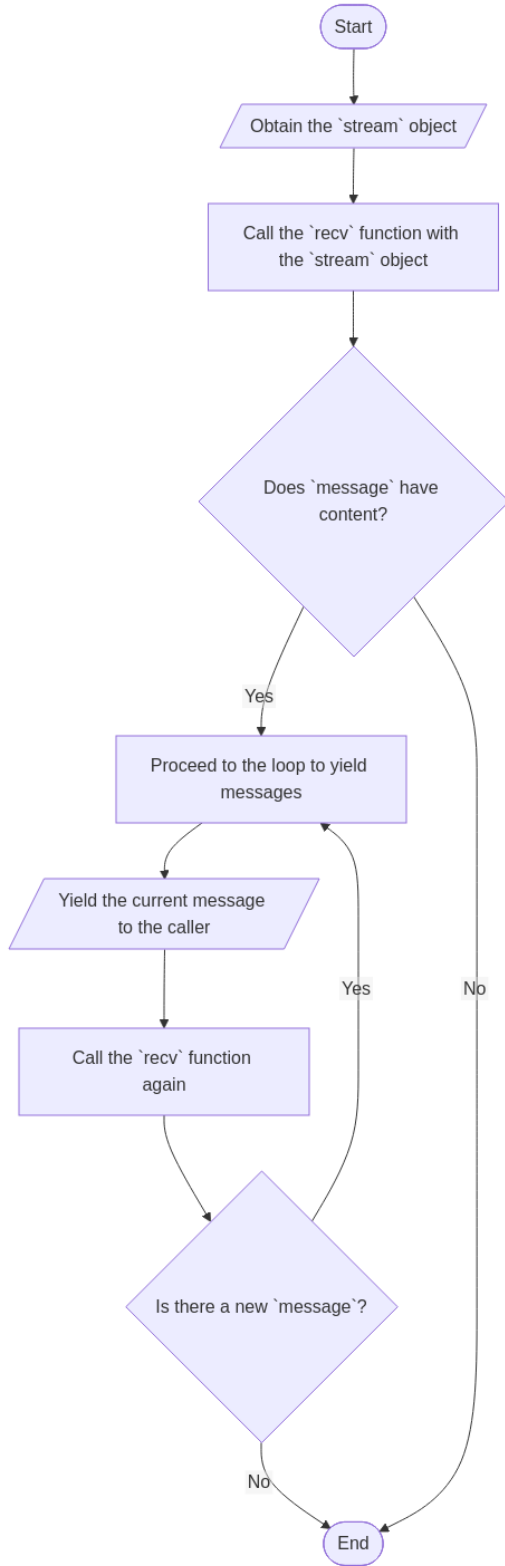
```
H1: source=Start
    target=Obtain the `stream` object
    condition=None
H2: source=Obtain the `stream` object
    target=Call the `recv` function with the
    `stream` object
    condition=None
H3: source=Call the `recv` function with the
    `stream` object
    target=Does `message` have content?
    condition=None
H4: source=Does `message` have content?
    target=Proceed to the loop to yield messages
    condition=Yes
H5: source=Is there a new `message`?
    target=Proceed to the loop to yield messages
    condition=Yes
H6: source=Proceed to the loop to yield messages
    target=Yield the current message to the caller
    condition=None
H7: source=Yield the current message to the caller
    target=Call the `recv` function again
    condition=None
H8: source=Call the `recv` function again
    target=Is there a new `message`?
    condition=None
H9: source=Is there a new `message`?
    target=End
    condition=No
H10: source=Does `message` have content?
    target=End
    condition=No
```

Triplet H5 demonstrates a correctly recovered curved back-edge. Because TRACE anchors its query at the arrowhead, curvature earlier along the connector does not change the local directional cue. The same mechanism handles overlapping or converging arrows: distinct detected arrowheads are queried independently, while multiple branches sharing an overlapping arrowhead yield the multiple-triplet output defined in Section 3.4.

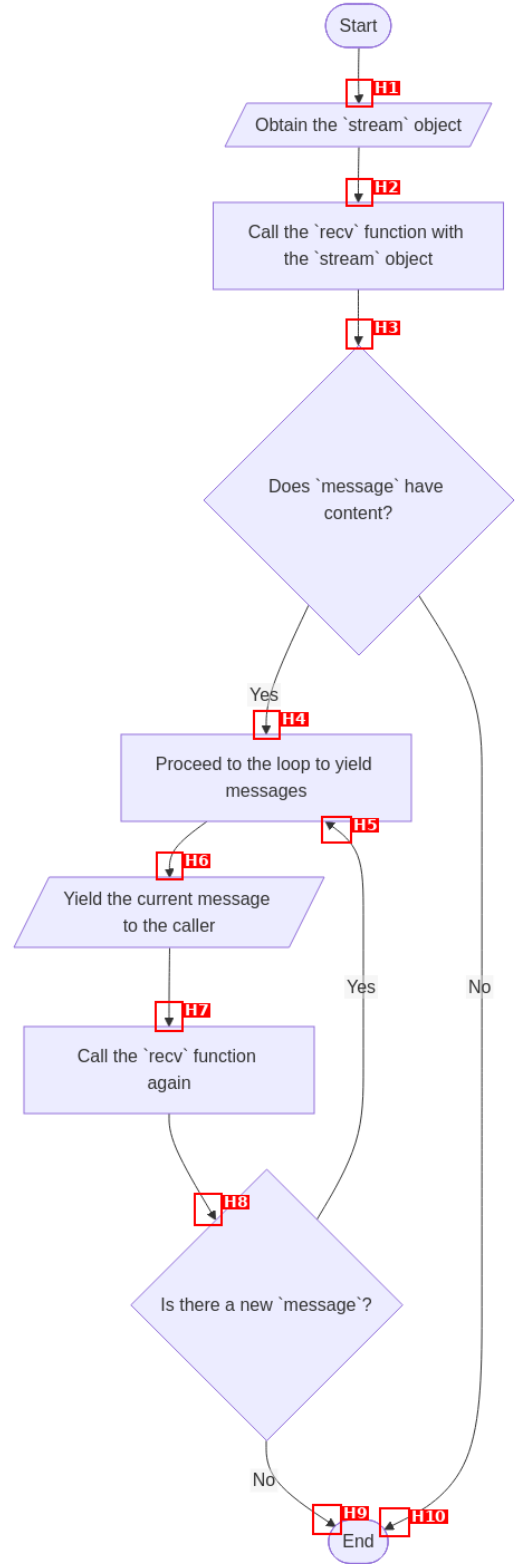
End-to-end output. The same ten triplets emitted as a flat list, with no link back to any arrowhead:

```
<Start, connectedTo, Obtain the `stream` object>
<Obtain the `stream` object, connectedTo,
  Call the `recv` function with the `stream` object>
<Call the `recv` function with the `stream` object,
  connectedTo, Does `message` have content?>
<Does `message` have content?, Yes,
  Proceed to the loop to yield messages>
<Is there a new `message`?, Yes,
  Proceed to the loop to yield messages>
<Proceed to the loop to yield messages, connectedTo,
  Yield the current message to the caller>
<Yield the current message to the caller, connectedTo,
  Call the `recv` function again>
<Call the `recv` function again, connectedTo,
  Is there a new `message`?>
<Is there a new `message`?, No, End>
<Does `message` have content?, No, End>
```

Why the format difference matters. In the TRACE output, each triplet is paired with an arrowhead label (H_k) that, via SAM 3, corresponds to a specific bounding box on the image. If H8 is judged incorrect by human reviewers, they can (i) highlight bounding box H8 on the image to localize



(a) Original FlowVQA flowchart code00005.



(b) SAM 3 detections on code00005: ten arrowheads labelled **H1–H10**, top-to-bottom and left-to-right.

Figure 3: FlowVQA flowchart code00005. (a) Original flowchart. (b) SAM 3 arrowhead detections, labelled **H1–H10**.

the error and (ii) re-issue the per-arrowhead query against *only* that arrowhead. Neither operation is

possible from the end-to-end output, where the only recourse is to re-run the whole-image model

and hope the new sample differs. This locality is also what underlies the per-arrowhead reweighting discussed in our Conclusion section.

F Implementation Details

All open-source models are trained and run locally on NVIDIA RTX 5880 Ada Generation GPUs; the two large-scale VLMs (GPT-5 and GLM-4.6V) are accessed through their official APIs. Our implementation is built on PyTorch (2.8.0, CUDA 12.8), with Hugging Face Transformers (4.57.1) and PEFT (0.15.2) for model loading and LoRA fine-tuning. Image preprocessing—diagram rasterization and highlight-marker overlay—uses Pillow (11.3.0) and OpenCV (4.11.0). The edit similarity behind relaxed F1 is computed with the python-Levenshtein library (0.27.3).

G Hyperparameters

Arrowhead reconstruction and highlight marker. For SVG sources (§3.2), we reconstruct each arrowhead using length $\ell = 10$ pixels and width $w = 10$ pixels, and expand its axis-aligned bounding box by padding $\rho = 5$ pixels. The per-arrowhead highlight marker overlaid on each resulting bounding box (§3.3) is a blue rectangle (RGB (0, 0, 255)) with a line width of 3 pixels. Figure 1 renders the markers in red for visual contrast against the figure background; the actual training and inference data use the blue color.

SAM 3 fine-tuning. We fine-tune SAM 3 (Carion et al., 2025) from its pretrained checkpoint on $\mathcal{D}_{det}$ using the prompt “arrowhead”. The detector loss combines L_1 and Generalized IoU (Rezatofighi et al., 2019) for bounding box regression with binary cross-entropy for object presence classification, matched bipartitely via the Hungarian algorithm (Carion et al., 2020) (cost weights: classification 2.0, box 5.0, GIoU 2.0). Detection hyperparameters are listed in Table 11.

YOLOv11 fine-tuning. For the ablation study (§4.3), we also fine-tune YOLOv11 (Ultralytics, 2024) on the same $\mathcal{D}_{det}$, treating “arrowhead” as a single object class. Training uses the Ultralytics CLI from the yolo11x pretrained checkpoint, with per-benchmark input resolution roughly matching each benchmark’s median image long-edge to avoid interpolation artifacts; all other training options follow Ultralytics defaults (SGD with momentum, default augmentation policy, AMP). For each

Hyperparameter	Value
Input resolution	1008 × 1008
Max annotations/image	50
LR (transformer)	8×10^{-5}
LR (vision backbone)	2.5×10^{-5}
LR scheduler	inverse-sqrt (warmup 20)
Layer decay (vision)	0.9
Weight decay	0.1
Epochs	30
Batch size	1 per GPU
Precision	bfloat16
Gradient clip	max norm 0.1

Table 11: SAM 3 fine-tuning hyperparameters.

Hyperparameter	Value
Model	YOLOv11x
Pretrained weights	yolo11x.pt (COCO)
Epochs	30
Global batch	8, 16, or 32 (grid)
GPUs	8
Input resolution	per-benchmark (640–2048)
Optimizer	SGD (Ultralytics default)
LR scheduler	linear (lr0=0.01, lr_f=0.01)
Augmentation	Ultralytics default
Precision	AMP
Early stopping	disabled (best by val mAP)

Table 12: YOLOv11 fine-tuning hyperparameters. Per-benchmark input resolution: FlowLearn 640, BPMN-VLM / FC_B 1280, FlowGen-m 1920, FlowVQA / FlowGen-e / FlowGen-h 2048.

benchmark we run a grid search over the per-device batch size ($\{1, 2, 4\}$ on 8 GPUs $\rightarrow$ global batch 8, 16, or 32) and select the checkpoint with the highest validation mAP. Hyperparameters: Table 12.

LoRA fine-tuning. Each of the five open-source backbones (§4.1.2) is adapted on $\mathcal{D}_{vlm}$ using LoRA (Hu et al., 2022) (Table 13). LoRA modules are applied to all linear layers of both the vision encoder and the language model; embeddings and the output head are frozen. Each backbone uses its native image preprocessing pipeline; for Qwen3-VL we set the token-based pixel range to 256–1280 × 28². Training uses DeepSpeed ZeRO Stage 3 (Rajbhandari et al., 2020). We perform an early-stopping grid search over batch sizes $B \in \{8, 16, 32, 64, 128\}$ with patience 1 on validation loss, and select the checkpoint with the lowest validation loss across all batch sizes. The validation set is the official split when available, otherwise a fixed-seed 10% partition of the training data.

Best LoRA configuration per benchmark. Tables 14 and 15 list, for each backbone, the batch size and epoch selected by the grid search on each

Hyperparameter	Value
LoRA rank r	32
LoRA alpha α	64
LoRA dropout	0.05
Target modules	all linear layers
Vision LoRA	enabled
Bias	none
Optimizer	AdamW
Learning rate	2×10^{-4}
LR scheduler	cosine
Warmup ratio	0.03
Weight decay	0.1
Epochs	10
Precision	bfloat16
GPUs	8
Per-device batch	1
Gradient accumulation	$B/8$
DeepSpeed	ZeRO Stage 3

Table 13: LoRA fine-tuning hyperparameters (shared across the five open-source backbones). Per-device batch is fixed at 1 across all 8 GPUs; gradient accumulation is set to match the effective global batch with B .

benchmark, for TRACE and the end-to-end baseline respectively.

Detection inference. At inference time, both SAM 3 and YOLOv11 filter their raw predictions by a confidence threshold before passing arrowheads to the VLM. SAM 3 uses a single score threshold of 0.3 across all benchmarks. YOLOv11’s confidence threshold is tuned per benchmark on the validation split since raw score calibration varies across benchmarks: FC_B 0.25, FlowLearn 0.30, FlowVQA 0.65, BPMN-VLM 0.30, FlowGen-e 0.30, FlowGen-m 0.30, FlowGen-h 0.20 (CBD and FC_A are absent because they lack ground-truth arrowhead bounding boxes and YOLOv11 is not fine-tuned for them). Each detector applies its own standard non-maximum suppression (NMS) with IoU threshold 0.7.

H Benchmark Statistics

Split protocol. CBD, FC_B, and BPMN-VLM have official train/val/test splits. For benchmarks with only train/test (FlowVQA, FlowLearn, FC_A), we partition each training set 9:1 into train and validation with a fixed seed. For FlowGen, we subsample 1,000 train and 500 test images per difficulty level, drawn uniformly from the four source formats (Mermaid, GraphViz, PlantUML, diagrams.net), and split the 1,000 further 9:1 into train and validation.

Restoring missing arrowhead-coordinate annotations for CBD and FC_A. CBD and FC_A are exceptions to the automatic synthesis pipeline because their public releases lack the fundamental arrowhead-coordinate annotations needed to construct $\mathcal{D}_{vlm}$ directly. We use SAM 3 to recover candidate boxes and manually verify their coordinates on the training and validation splits. This manual effort does not create new task annotations; it only restores the arrowhead coordinates absent from the released data. No node, condition, edge relation, or triplet label is manually supplied or corrected. Verification covers 300 training and 106 validation images for CBD, and 191 training and 21 validation images for FC_A; approximately 30% of the images in each benchmark require bounding-box correction. Because the recovered boxes are needed only for model training and validation, this procedure is not applied to the test split.

Per-benchmark statistics. Table 16 reports source formats and image-level splits.

Licenses. FlowVQA: MIT. FlowLearn: CC-BY-NC-SA 4.0. CBD: research use only. FC_A (OHFCD): CC-BY-NC-SA 3.0. FC_B: no license attached on the source page (research use). BPMN-VLM: research use only (inherited from the upstream Camunda bpmn-for-research corpus, which permits academic/research use and forbids commercial use). FlowGen-e/m/h: Apache-2.0. We use each benchmark only for academic evaluation within the limits of its stated terms.

I Triplet Normalization

Both F1 and rF1 (§4.1.4) compare predicted to ground-truth triplets after a normalization pass applied independently to each component (source, condition, target). The full sequence is:

1. Unicode NFKC normalization (full-width $\rightarrow$ ASCII, decomposed forms);
2. Stripping decorative brackets;
3. Math symbol mapping ($\leq \rightarrow <=$, $\neq \rightarrow !=$, $\times \rightarrow *$, $:= \rightarrow =$);
4. Whitespace cleanup (newlines $\rightarrow$ spaces, no spaces around operators, collapse multi-space);
5. ++/– spacing ($i + + \rightarrow i++$);
6. Lowercasing.

	Qwen3-VL-4B		Qwen3-VL-8B		Gemma-3-4B-IT		LLaVA-v1.6-7B		MiniCPM-V-4.5-8B	
Benchmark	Batch	Epoch	Batch	Epoch	Batch	Epoch	Batch	Epoch	Batch	Epoch
FlowVQA	64	2	128	2	64	5	64	8	128	3
CBD	64	1	32	1	32	1	64	2	64	1
FC_A	16	8	32	9	32	6	32	4	64	5
FC_B	16	1	32	1	64	4	64	2	32	2
FlowLearn	128	3	128	3	128	5	128	8	128	3
BPMN-VLM	64	7	16	5	16	4	64	5	32	6
FlowGen-e	64	7	64	6	32	2	128	1	64	4
FlowGen-m	128	5	128	3	32	3	128	2	32	4
FlowGen-h	128	5	128	7	64	2	128	1	32	4

Table 14: Selected LoRA batch size and best epoch for TRACE’s VLM on each benchmark.

	Qwen3-VL-4B		Qwen3-VL-8B		Gemma-3-4B-IT		LLaVA-v1.6-7B		MiniCPM-V-4.5-8B	
Benchmark	Batch	Epoch	Batch	Epoch	Batch	Epoch	Batch	Epoch	Batch	Epoch
FlowVQA	64	3	32	2	32	2	8	1	64	2
CBD	8	2	8	2	8	2	32	2	16	2
FC_A	16	4	8	2	16	3	8	4	16	3
FC_B	8	3	8	2	16	3	32	3	16	3
FlowLearn	16	2	32	2	32	2	32	2	32	2
BPMN-VLM	8	3	8	3	8	3	16	2	32	5
FlowGen-e	32	6	16	4	16	2	16	2	16	2
FlowGen-m	16	4	32	3	16	2	16	2	16	3
FlowGen-h	16	5	32	5	16	3	8	2	16	3

Table 15: Selected LoRA batch size and best epoch for the end-to-end baseline on each benchmark.

Benchmark	Source	Train	Valid	Test
FlowVQA	Mermaid	1,187	132	953
FlowLearn	Mermaid	7,200	800	2,000
CBD	XML	300	106	96
FC_A	INKML	191	21	145
FC_B	INKML	280	196	196
BPMN-VLM	BPMN-XML	100	50	51
FlowGen-e	mixed	900	100	500
FlowGen-m	mixed	900	100	500
FlowGen-h	mixed	900	100	500

Table 16: Per-benchmark statistics. *Source*: textual format of each benchmark’s source files, used by the data-synthesis pipeline (§3.2). Train/Valid/Test counts are at the image level. FlowGen’s *mixed* source pools Mermaid, GraphViz, PlantUML, and diagrams.net uniformly.

Relations are additionally mapped through a small synonym table (e.g., yes/y $\rightarrow$ yes, blank $\rightarrow$ connect).

J Prompt Templates

This appendix lists the prompts used by the two paradigms: one whole-image prompt for the end-to-end baseline, and three per-arrowhead prompts for TRACE (one for generic flowcharts, one for BPMN diagrams with pool/lane hierarchy, one for Flow-

Gen flowcharts with explicit groups/containers).

End-to-end (whole image). Used by all end-to-end baselines across the nine benchmarks. The model is given the whole image and asked to enumerate every triplet in one pass, with a fixed example block to constrain the output format.

```
[System Input]
(none)

[User Input]
{image input: flowchart}
{text input:
Please describe all the information in the flowchart image
in the form of triplets: For each arrow labeled
with X directed from node A to node B, output a
triplet: <A, X, B>; For each unlabeled arrow
directed from node A to node B, output a triplet: <A,
connectedTo, B>; For each node A fully inside node
group B, output a triplet <A, partOf, B>.

The following triplets are example outputs:
<Food Packaging Improvement, secures, Quantum Computing
Integration>
<Graphene Production, catalyzes, Nanosensors>
<Graphene Production, partOf, Drug Delivery Systems>
<Nanocomposites, connectedTo, Spectroscopy>
<Nanodevice Fabrication, connectedTo, Graphene Production>
<Nanosensors, connectedTo, Nanocomposites>
<Spectroscopy, educates, Nanodevice Fabrication>
}
```

Per-arrowhead, generic flowchart. Used by TRACE on FlowVQA, FlowLearn, CBD, FC_A, and FC_B. The model is given an image with one

arrowhead highlighted by a blue bounding box, and asked to emit three aligned output lines. Multi-branch arrowheads use “| |” as a separator.

```
[System Input]
(none)

[User Input]
{image input: flowchart}
{text input:
# Role: Flowchart Connectivity Analyst

## Profile
- **Task:** Analyze a flowchart image where a
specific **arrowhead** is highlighted by a
**Blue Bounding Box**.
- **Goal:** Accurately determine the arrow's
direction and trace its path to identify the
connected **Source Node(s)**, **Target Node(s)**,
and any **Condition Label(s)**.

## Analysis Steps (Rules)
1. **Locate Focus:**
- Strictly focus on the arrowhead enclosed
within the **Blue Bounding Box**.
2. **Determine Direction:**
- Observe the orientation of the arrow inside
the blue box to determine its flow direction.
3. **Trace Backwards (Find Source):**
- Visually trace the line **backwards** to find
the Originating Shape(s).
- **Extract Text:** Read the text inside.
4. **Check for Condition Label(s) - Strict
Alignment:**
- Look for text labels along the arrow line
(e.g., "Yes", "No").
- **One-to-One Mapping:** You must identify the
condition status for **EVERY** path
identified in Step 3.
- **Placeholder Rule:** If a specific path does
NOT have a condition label, you must
explicitly record it as **"None"**.
- *Example:* If there are two paths, one with
"Yes" and one without a label, output:
`Yes || None`.
5. **Trace Forwards (Find Target):**
- Visually trace the arrow **forwards** to find
the Destination Shape(s).
- **Extract Text:** Read the text inside.
6. **Content Handling:**
- **Transcription:** Keep original
spelling/formatting.
- **Separator:** Use ` ` || ` ` to separate
contents for multiple paths.

## Output Format
- Always provide all three lines.
- Ensure the number of items separated by `||`
is consistent across all three lines to
maintain alignment.

source_node: Text from Source A || Text from Source B
target_node: Text from Target A || Text from Target B
condition: Condition for A || Condition for B
}
```

Per-arrowhead, BPMN. Used by TRACE on BPMN-VLM. Identical to the generic prompt except the model also reports the pool/lane that contains each endpoint, yielding five output lines.

```
[System Input]
(none)

[User Input]
{image input: BPMN diagram}
{text input:
# Role: BPMN Connectivity Analyst
```

```
## Profile
- **Task:** Analyze a BPMN diagram image where a
specific **arrowhead** is highlighted by a
**Blue Bounding Box**.
- **Goal:** Identify the arrow's source node, its
containing pool/lane, any condition label, the
target node, and its containing pool/lane.

## Analysis Steps
1. **Locate Focus:** Strictly focus on the
arrowhead enclosed within the **Blue Bounding
Box**.
2. **Determine Direction:** Observe the arrow
orientation to determine flow direction.
3. **Trace Backwards (Find Source):** Follow the
arrow line backwards to find the originating
shape. Read the text inside. Also identify
which pool or lane (the most immediate
container) it belongs to.
4. **Check for Condition Label:** Look for a text
label along the arrow line (e.g., "Yes", "No",
"approved"). If none exists, record **"None"**.
5. **Trace Forwards (Find Target):** Follow the
arrow forward to find the destination shape.
Read the text inside. Also identify which pool
or lane it belongs to.

## Output Format
Always provide exactly five lines in this order:

source_node: <text inside the source shape>
source_location: <pool or lane name that directly
contains the source, or None>
condition: <label on the arrow, or None>
target_node: <text inside the target shape>
target_location: <pool or lane name that directly
contains the target, or None>
}
```

Per-arrowhead, FlowGen. Used by TRACE on FlowGen-e/m/h. Like the BPMN variant, but the location field refers to the immediate enclosing group or container box (FlowGen sources support explicit grouping). The variant also supports “| |” for multi-branch arrowheads (the BPMN variant does not).

```
[System Input]
(none)

[User Input]
{image input: flowchart}
{text input:
# Role: Flowchart Connectivity Analyst

## Profile
- **Task:** Analyze a flowchart image where a
specific **arrowhead** is highlighted by a
**Blue Bounding Box**.
- **Goal:** Identify the arrow's source node, its
containing group (if any), any condition/edge
label, the target node, and its containing
group (if any).

## Analysis Steps
1. **Locate Focus:** Strictly focus on the
arrowhead enclosed within the **Blue Bounding
Box**.
2. **Determine Direction:** Observe the arrow
orientation to determine flow direction.
3. **Trace Backwards (Find Source):** Follow the
arrow line backwards to find the originating
shape. Read the text inside. Also identify
which group or container (the most immediate
bounding box with a label) it belongs to,
if any.
4. **Check for Condition Label:** Look for a text
label along the arrow line (e.g., "Yes", "No",
```

```

    "approved"). If none exists, record "None"**.
5. **Trace Forwards (Find Target):** Follow the
   arrow forward to find the destination shape.
   Read the text inside. Also identify which
   group or container it belongs to, if any.
6. If the blue bounding box covers multiple
   arrowheads, use " || " to separate multiple
   values in each field.

## Output Format
Always provide exactly five lines in this order:

    source_node: <text inside the source shape>
    source_location: <group or container name that
                    directly contains the source, or None>
    condition: <label on the arrow, or None>
    target_node: <text inside the target shape>
    target_location: <group or container name that
                    directly contains the target, or None>
}

```

K Detection-reconstruction Baseline Output Mapping

Both detection-reconstruction baselines emit primitive detections (shape boxes plus arrow detections with endpoint information) rather than the (s, c, t) triplet format used by our F1/rF1 evaluator. We map their outputs to triplets faithfully to each system’s own assembly logic; the two pipelines are described below.

BlockNet. BlockNet’s local information extractor consists of a YOLOv5 detector plus a graph-assembly procedure. We run the released best-checkpoint weights and reuse the assembly procedure verbatim, with two faithful substitutions: (i) we replace the now-deprecated Pororo OCR with EasyOCR (English), since CBD and FC_B are English-only; (ii) the rest of the pipeline (NMS, contact-point matching, head/tail disambiguation) is unchanged. Class mapping: $\emptyset \rightarrow$ arrow (becomes an edge); $\{1, 2, 3, 5\} \rightarrow \{\text{terminator, process, decision, data}\}$ (become nodes); other classes are discarded. For each arrow, the two endpoints are matched to the closest contact point (top / bottom / left / right) of any node; the arrowhead direction is determined by the half of the arrow bounding box with the higher white-pixel count, fixing the (source, target) assignment. The condition is filled with OCR words that are not absorbed by any node, attached to the spatially nearest arrow; arrows with no remaining OCR words receive an empty condition.

Flowmind2digital. Flowmind2digital uses a Detectron2-based detector with keypoint heads. It emits 12 classes: shapes {circle, diamond, long_oval, hexagon, parallelogram, rectangle, trapezoid, triangle} (classes

0–7), a generic text class (8), and three relation types {arrow, double_arrow, line} (9–11). Connections are built from the predicted arrow keypoints (head / tail), which select the source and target shapes. Text is assigned in two passes: (i) every text-class box that is fully contained in a non-text shape’s box is “owned” by that shape; (ii) the owned text region (or, as a fallback, the shape’s own box) is fed to PaddleOCR and the recognized string becomes the shape’s label. Each connection then yields a triplet whose source/target texts are the OCR strings of the two endpoint shapes, and whose *condition* is the predicted arrow class name (arrow, double_arrow, or line).

L Tool-call Schema for QA

Setup. Under **TRACE + tool-call** (§4.5), the Qwen3-VL-4B reasoner is given the flowchart image, the question, and access to ten deterministic graph query tools that operate on a directed multi-graph built from TRACE’s extracted triplets. Node identity uses a normalized lowercase form (punctuation stripped, whitespace collapsed) so light formatting differences do not mis-duplicate nodes. The reasoner follows the standard tool-call loop: it emits `<tool_call>{...}</tool_call>` blocks, the runner dispatches each call against the graph, the JSON results are appended as `role=tool` messages, and the loop repeats until the model emits a final answer or the step budget (5) is exhausted.

System prompt. The reasoner is initialized with the following system message:

```

You are an expert assistant for flowchart question
answering.
You have access to tools that query a graph extracted from
the flowchart (nodes are text boxes; edges are
arrows; some edges have a condition label).
When a question is best answered with graph facts (counts,
neighbors, edge existence, shortest path, what
comes next/before, etc.), CALL TOOLS first, then
answer.
Node labels in the graph may differ slightly from the
question's wording -- use `find_node` to resolve
ambiguous names before passing them to other tools.
Use at most a handful of tool calls per question (no loops
).
When you are ready to answer, give a SHORT plain-text
response. Do NOT wrap the final answer in <tool_call
> tags or JSON.

```

Tool list. Signatures and short docstrings (parameters in OpenAI tool-schema JSON):

```

count_nodes() -> int
# Total number of distinct nodes.
count_edges() -> int
# Total number of directed edges.
list_nodes() -> list[str]
# Every node text in the graph.
list_edges() -> list[{source, end, condition}]
# Every directed edge.
find_node(query: str, top_k: int = 5)
-> list[str]
# Fuzzy-search for nodes whose text matches
# the query. Used to resolve the canonical
# label before calling other tools.
get_outgoing(node: str)
-> list[{end, condition}]
# All outgoing edges from `node`.
get_incoming(node: str)
-> list[{source, condition}]
# All incoming edges into `node`.
get_neighbors(node: str)
-> list[str]
# Undirected neighbors (union of in + out).
has_edge(source: str, end: str,
    directed: bool = True) -> bool
# Whether an edge from `source` to `end`
# exists. directed=false allows either way.
find_path(source: str, end: str,
    directed: bool = True)
-> list[str] | None
# A shortest path from `source` to `end`.

```

M LLM-as-judge for QA Evaluation

Setup. FlowVQA answers in §4.5 are scored by GLM-4.6V (Zhipu AI, 2025) via the Z.ai OpenAI-compatible API. For each prediction we query the judge once per reference label (FlowVQA provides three references A1/A2/A3 per question); the prediction counts as correct if *any* of the three is judged *Correct*, matching TextFlow’s protocol. The judge runs at temperature 0 (deterministic), max_tokens=8 (just “Correct” / “Incorrect”), thinking mode disabled, with 10 concurrent requests.

Judge prompt. Verbatim from TextFlow’s released evaluation code; {q}, {a}, {gt} are filled in with the question, the model’s predicted answer, and one reference label.

```

Task: Verify if the provided answer is correct based on
the given ground truth.

You are given a question, an answer and the ground truth.
Your task is to determine whether the provided
answer matches the ground truth. Output "Correct" if
the answer matches ground truth, otherwise output "
Incorrect".

Question: {q}

Answer: {a}

Ground Truth: {gt}

```